MLOps Portfolio: Complete Study and Interview Guide

Updated 29 October 2026. Architecture, fundamentals, worked examples and interview preparation aligned with the execution audit.

MLOps Portfolio: Tools and Concepts Primer

Updated 29 October 2026. Learn these concepts in order, then connect each one to a file and an observed result. This project uses Python and infrastructure configuration, with no JavaScript frontend.

1. MLOps and the difference between a model and a system

A model maps inputs to predictions. A working system must also load the right artifact, accept valid inputs, survive failures and report whether it is ready. MLOps applies software and operational practices to the model lifecycle. This repository demonstrates some of those practices; it does not implement every stage of production MLOps.

> Analogy: a recipe is the model. A restaurant also needs a kitchen, orders, reliable equipment and a way to notice failures. A good recipe cannot compensate for lost orders or an unavailable oven.

Study order: understand the 40-message fixture and classifier, run Python serving, follow one Kafka record, inspect the Kubernetes probes, then use Terraform against LocalStack. Treat Bedrock as an optional, unverified live integration.

2. Python modules, environments and dependency versions

A module is a Python source file. A virtual environment gives one project its own installed packages. The serving image uses Python 3.11; CI also checks Python 3.12. Install the requirements for the modules you run. Test and PDF dependencies are separate from the serving requirements.

The audit found that `kafka-python==2.0.2` could not import on Python 3.12. The repaired requirements use 2.2.21. Model training also pins NumPy and SciPy alongside scikit-learn. A dependency freeze in the evidence records the actual resolved environment; selected version pins do not lock every transitive package or Docker base-image byte.

> Analogy: each environment is a separate toolbox. Writing down which tools you used makes a repair repeatable. Merely saying "use a toolbox" does not identify compatible tool sizes.

3. Classification, labels and the fixture

Classification chooses a category. Here the labels are `ham` and `spam`, with 20 examples of each. Forty curated messages are useful for running an operations demo quickly. They are too small and unrepresentative to establish production message-filter quality.

Accuracy is the fraction of all predictions that match the label. Precision asks how many predicted-spam messages are actually spam. Recall asks how many actual spam messages were found. F1 balances precision and recall. The cost of missing spam can differ from the cost of hiding a legitimate message, so accuracy alone cannot choose an operating policy.

4. TF-IDF and sparse matrices

Computers need numbers, so the vectorizer creates one feature position per learned token. TF describes occurrence within a message; IDF reduces the weight of terms common across the training corpus. English stop words are removed, and text is lowercased. The resulting matrix is sparse because a short message uses only a small part of the vocabulary.

> Analogy: imagine a catalogue with a column for every product. One receipt mentions only a few products, so writing thousands of explicit zeroes would waste space. Sparse storage records the useful entries.

The vocabulary and IDF are fitted data. They must be learned inside each training fold. Unseen terms do not create new feature columns during inference. If a message has no known tokens, the API includes a warning; the classifier can still return an intercept-driven result.

5. Logistic regression, probabilities and Pipelines

Logistic regression forms a weighted sum of features and converts it into class probabilities. Its regularisation limits the size of learned weights. A linear classifier is a sensible baseline for sparse text, but the small fixture remains the main limitation here.

For a hypothetical example, an intercept of -1 and a prize feature contributing 1.6 give a score of 0.6. The sigmoid calculation, `1 / (1 + exp(-0.6))`, gives about 0.646. These are teaching numbers, not fitted coefficients from this project.

The scikit-learn Pipeline joins the fitted TF-IDF transform and classifier. `fit` learns; `predict` selects a label; `predict_proba` returns model probabilities. The API's `confidence` is their maximum, rounded to four decimals. It has not been calibrated against representative held-out data.

`joblib.dump` stores the fitted Pipeline. `joblib.load` restores it for serving. Keep versions compatible and only load trusted artifacts. A saved model file is not automatically safe, portable across arbitrary library versions or tracked in a model registry.

6. Cross-validation and leakage

Stratified five-fold validation preserves approximate class proportions. Each run trains on 32 messages and tests on eight. The entire Pipeline is fitted separately each time, so held-out text cannot teach the vectorizer its vocabulary or IDF.

The measured fold accuracies are 87.5%, 87.5%, 62.5%, 87.5% and 62.5%. Their mean is 77.5%; population standard deviation across these folds is 12.25 percentage points. The final artifact is then fitted on all 40 records and reaches 100% training accuracy. Those numbers answer different questions.

> Analogy: practising all the answers can produce a perfect practice score. Testing on questions left out of practice gives a better check, but eight questions still make a fragile examination.

There is no independent real-world holdout, hyperparameter-selection protocol or probability calibration study. The five-fold result should be described as a fixture diagnostic. Fold standard deviation is not a confidence interval.

7. HTTP, FastAPI, Uvicorn and validation

HTTP lets one program send a request and receive a response. FastAPI defines the routes and request contract; Uvicorn runs the application as an ASGI server. Pydantic validates JSON against the `Message` model before prediction logic receives it.

ASGI is the interface through which the server calls the Python application; it is not a separate prediction model.

Request/result	Meaning here
`GET /health -> 200`	Process can answer
`GET /ready -> 200`	Model loaded successfully
`GET /ready -> 503`	Model unavailable
`POST /predict -> 200`	Valid request; classifier result
`POST /predict -> 422`	Request violates the text schema
`POST /predict -> 503`	Model unavailable

The application lifespan handler loads once per process. It preserves fast liveness while readiness reflects the actual model state. A missing artifact requires training and a service restart. There is no automatic model refresh or background retraining.

8. Docker and Compose networking

An image is a packaged filesystem plus runtime instructions; a container is a running instance. The serving Dockerfile installs dependencies before copying source to improve build-cache reuse, trains at build time and starts Uvicorn bound to `0.0.0.0` inside the container.

`EXPOSE 8000` documents the container port. `-p 8767:8000` maps host port 8767 to it. Compose gives services a shared network and names such as `kafka` and `spam-api`. Inside another container, localhost means that container itself.

Kafka listeners bind sockets; advertised listeners tell clients where to reconnect after receiving broker metadata. This Compose stack advertises `kafka:9092`, appropriate for its containers. A host port mapping does not make that advertised hostname valid for every host client. Health-based `depends_on` waits for readiness, rather than merely container creation.

9. Kafka topics, partitions, groups and offsets

A topic is a named log, split into partitions. Each record has a position called its offset. Ordering is per partition. A consumer group distributes partitions among consumers; adding consumers beyond the partition count does not create additional active work for that topic.

The project writes inputs to `messages` and results to `predictions`. A committed offset names the next record to read after restart. `auto_offset_reset=earliest` applies when there is no valid committed offset; it does not force every restart to replay the whole topic.

KRaft lets Kafka's own controllers manage cluster metadata using the Raft consensus protocol, instead of a separate ZooKeeper service. This demo combines controller and broker roles on one node; it does not demonstrate a resilient controller quorum.

> Analogy: a partition is an ordered notebook and an offset is a page number. Committing is moving a bookmark. Moving it before you finish copying the result can make an unfinished page look completed.

The consumer disables background commits, waits on the producer's send future and then commits the exact next input offset. Acknowledgement failure leaves the input uncommitted. A crash between successful output and input commit can still duplicate output, so the guarantee is

at least once. Stable event IDs support a future deduplicator, which is not implemented.

`acks=all` waits for the broker's required in-sync acknowledgements: confirmations from replicas considered sufficiently caught up with the leader. With one broker and replication factor one, it still means only one copy. This demonstration has no multi-broker durability test, transactions, dead-letter topic or automatic poison-record recovery.

10. Kubernetes reconciliation and probes

A Pod runs containers; a Deployment manages replaceable replicas; a Service selects matching pods and gives callers a stable network target. The manifests request two API replicas and define CPU/memory requests and limits.

The startup probe allows model loading. Readiness checks `/ready` to decide whether to route traffic. Liveness checks `/health` to decide whether the process should be restarted. These probes ask different questions. The integration test checks two ready replicas and replacement after deleting one pod; that is not a cross-node disaster-recovery benchmark.

11. Terraform, state and LocalStack

Terraform describes desired resources in HCL and uses provider plugins to call APIs. `init` installs providers, `plan` previews changes, `apply` performs them and `destroy` removes the selected resources. State records Terraform's association between configuration and resource identities; it must be protected because it can contain sensitive values.

LocalStack emulates AWS APIs locally. The supplied configuration uses dummy credentials and local endpoints for an S3 bucket and a DynamoDB table. The table uses string `id` as its key and on-demand billing mode. Neither resource is consumed by the serving or streaming application yet.

The acceptance check applies both resources, verifies them through the emulator, checks for an empty second plan and destroys the isolated test resources. It proves an emulated lifecycle, not production AWS behavior or a deployed Kubernetes cluster created by Terraform.

12. Bedrock, interpretation and the limits of the LLM layer

Boto3 calls the Bedrock runtime API using credentials supplied externally. The CLI first requests the classifier's real label and score, then builds the optional LLM request. Credentials and model access are necessary for a live call; mocked SDK tests do not provide that access or prove a live explanation's quality.

The LLM sees the message and prediction, not the classifier's measured feature contributions. Its explanation is therefore a possible interpretation. A faithful attribution exercise would inspect the vectorizer features and model coefficients or use an appropriate attribution method. Asking an LLM "why" does not create that evidence.

MLOps Portfolio: How It Works

Updated 29 October 2026. This guide describes the repaired Python implementation and the execution audit. It replaces the earlier guide's claims about lazy loading, integrated cloud storage and a completed Bedrock demonstration.

1. Explain the project accurately

The project packages a small English spam classifier as a Python HTTP service, sends messages through Kafka for inference, and supplies separate infrastructure exercises. It is an operations demonstration built around a TF-IDF and logistic-regression pipeline. The dataset contains 40 curated examples, not a representative production corpus.

There are five modules. Three reuse the classifier: local serving, Kafka inference and a Kubernetes deployment of the serving image. Terraform separately creates an S3 bucket and a DynamoDB table in LocalStack. These resources are not wired into the application. The optional Bedrock command can ask an LLM to interpret an actual API prediction, but a live AWS invocation is not part of the verified result.

> Analogy: a small practice restaurant has a working kitchen, an order conveyor and a way to open two serving counters. A separately built storeroom exists, but the kitchen does not automatically use it just because it is on the same blueprint.

2. Follow the code and the boundaries

Module	Actual responsibility	Important boundary
`serving/train.py`	Evaluate the fixture, fit the full pipeline, save model and metadata	Evaluation uses held-out folds; the final serving model uses all 40 records
`serving/app.py`	Load the model at startup; expose health, readiness and prediction	No authentication, drift monitor or online retraining
`kafka-streaming/`	Produce messages; call HTTP inference; publish confirmed outputs	At-least-once processing; duplicates remain possible
`kubernetes/`	Run two serving replicas with startup/readiness/liveness probes	Local-cluster demonstration, not a production cloud installation
`terraform-localstack/`	Create an S3 bucket and DynamoDB table in an emulator	No application artifact upload or prediction persistence
`bedrock-explainer/`	Request an LLM interpretation using the API's actual label and score	Requires real AWS access; interpretation is not feature attribution

The root README and `docs/verification.md` record which checks actually ran. The existence of a Dockerfile or YAML manifest alone does not prove a deployment worked.

3. Understand the data and model

The CSV has 40 unique messages: 20 `ham` and 20 `spam`. Its text looks like manually curated demonstration examples; no external benchmark provenance is supplied. The audit does not relabel it as real-world SMS data. There are no exact duplicate texts in this fixture.

`TfidfVectorizer` lowercases text, removes its English stop-word list and learns vocabulary and document-frequency weights. A word that appears everywhere is less informative than one concentrated in a few messages. The resulting sparse matrix feeds logistic regression, which

learns weights for a binary decision.

> Analogy: TF-IDF turns each message into a shopping receipt of weighted words. Logistic regression adds up the evidence on that receipt and compares the two possible labels.

A scikit-learn `Pipeline` stores the vectorizer and classifier together. Saving only the classifier would lose the mapping from words to feature positions. `joblib` persists the fitted pipeline; load only an artifact you trust, because pickle-based formats can execute code during deserialization.

4. Separate evaluation from final fitting

The repaired training command runs stratified five-fold evaluation with seed 42. For each fold, the vectorizer and classifier fit only that fold's 32 training messages, then predict the remaining eight. The vocabulary is not fitted once across all 40 before validation.

Fixture evaluation	Measured value
Five fold accuracies	87.5%, 87.5%, 62.5%, 87.5%, 62.5%
Mean accuracy	77.50%
Accuracy standard deviation across folds	12.25 percentage points
Mean spam precision	86.67%
Mean spam recall	65.00%
Mean spam F1	0.7429
Full-fixture training accuracy	100.00%

The standard deviation is fold dispersion, not a confidence interval. These are tiny-fixture diagnostics and do not establish performance on real inboxes. One separate audit-only 32/8 split happened to score 100%; it is not selected as the headline result. That contrast shows how unstable small samples can be.

After evaluation, training fits a fresh pipeline to all 40 records for the serving artifact. `model_metadata.json` records class counts, data/model hashes, seed, scikit-learn version and fold metrics. This report describes the artifact build; it is not a model registry or a deployed monitoring system.

5. Trace an HTTP prediction

The FastAPI lifespan handler loads and checks the trusted model once during process startup. Each replica has its own model instance. The old lazy-load path has been replaced, so the first prediction no longer owns the responsibility for loading the artifact.

`GET /health` reports that the process can answer HTTP. `GET /ready` requires a successfully loaded model. If the artifact is missing or corrupt, the process can remain alive while readiness and prediction return 503. Train the artifact and restart the service to recover; there is no background reload loop.

`POST /predict` accepts exactly one string field, `text`. Blank strings, whitespace-only text, non-string values, extra fields and inputs over 10,000 characters receive 422. The original string is preserved. A text with no known vocabulary tokens still receives the classifier's prior-driven output, with `warning: no_known_tokens` so the limitation is visible.

The response includes text, label, maximum class probability under the legacy field name `confidence`, a model-hash prefix, and an optional warning. That probability is not a calibrated guarantee of correctness. A 0.66 score does not establish that 66 out of 100 similar production messages will be classified correctly.

> Analogy: liveness asks whether the restaurant's phone works. Readiness asks whether the kitchen is equipped to accept an order. A working phone should not make a missing kitchen look ready.

6. Trace one Kafka record

The producer publishes JSON to `messages`. The consumer validates the outer message shape and calls the model API. It retries connection failures, timeouts, 429 and server errors a bounded number of times. A permanent 4xx request failure propagates immediately instead of being retried 40 times.

For input partition 2, offset 17, it constructs `event_id = messages:2:17`. It publishes the prediction with this stable key, waits for the broker acknowledgement through the send future, and only then commits input offset 18 for that specific partition. Kafka committed offsets identify the next record to read.

The old code enabled background offset commits and never inspected the individual publication future. Fault injection showed that an output failure could pass through without being noticed. Regression checks now prove that prediction, send or acknowledgement failure cannot call input commit.

This is at-least-once processing, not exactly-once delivery. If publication succeeds and the process crashes before committing offset 18, restarting can publish the same event again. A downstream deduplicator could use the stable event ID, but no such storage/deduplication implementation exists here. The single-node broker also has replication factor one.

Malformed JSON, invalid input or a persistent API failure stops the consumer without acknowledging the failed record. That preserves retryability, but a poison record can block progress until an operator fixes or handles it. There is no dead-letter queue or automatic recovery supervisor in this demonstration.

7. Containers and orchestration

The serving Dockerfile installs the supported dependencies, copies source and data, trains during the image build and runs Uvicorn on container port 8000. Local examples publish that as host port 8767. `EXPOSE` documents a port; `-p` actually maps it.

Compose starts a real single-node Kafka broker in KRaft mode. Clients inside the network use the advertised address `kafka:9092`; that address is not a ready-made host-client listener. Health-based dependencies keep the inference consumer from starting before the broker and API are ready.

Kubernetes runs two replicas of the same serving image. A startup probe gives model loading time, readiness uses `/ready`, and liveness uses `/health`. The Service selects pods by labels. The acceptance workflow waits for two ready replicas, performs HTTP predictions, deletes a pod and verifies that the Deployment restores capacity.

Containers improve packaging consistency. Floating base-image tags and unresolved transitive dependencies still prevent a promise of bit-for-bit identical rebuilds forever. No throughput, latency SLO, cross-node failover or production capacity claim follows from these functional checks.

8. Terraform and Bedrock

Terraform's locked AWS provider targets LocalStack endpoints with dummy `test` credentials. The lifecycle is init, validate, plan, apply, inspect resources, check a second plan for no changes, then destroy the isolated resources. The S3 bucket is `mlops-model-artifacts`; the on-demand DynamoDB table is `predictions`, keyed by string `id`.

These are provisioned resources, not evidence that the API stores predictions there. LocalStack success is also not an AWS deployment result. The supplied application reads its model from the container filesystem.

The Bedrock CLI now gets the label and score from `/predict`, rather than inventing a 68% example. The request explicitly asks for possible surface clues and identifies its answer as an LLM interpretation. Request-shape tests use a mocked SDK response. No AWS credentials, billable invocation or live Bedrock quality evaluation is included in the passing evidence.

9. Reproduce and explain the result

Start with the root README's Python setup or serving Docker commands. Run `pytest tests -q`, then the integration commands in the module READMEs. CI separates Python regression checks from actual Kafka, Terraform/LocalStack and Kubernetes jobs; a unit-test pass is not substituted for a service integration pass.

For an interview, explain three concrete improvements: model-aware readiness, acknowledge-after-publication streaming, and training-fold-only evaluation. Then state the limits: 40 curated messages, no production accuracy claim, optional unverified Bedrock, independent cloud resources, no monitoring/retraining or exactly-once pipeline.

MLOps Portfolio: Interview Questions

Updated 29 October 2026. Read the tools primer and architecture guide first. These answers describe the repaired code and measured evidence; adapt the wording to your own understanding.

1. Start with the project

Interviewer: What did you build?

You: I built a Python MLOps demonstration around one TF-IDF and logistic-regression spam classifier. It has a FastAPI serving image, a Kafka inference flow and Kubernetes manifests. A separate Terraform exercise creates S3 and DynamoDB resources in LocalStack. There is also an optional Bedrock interpretation command, whose live AWS path remains unverified.

Interviewer: Is this a production spam detector?

You: The engineering paths are testable, but the model uses only 40 curated messages. I describe it as an operations demonstration, with a small fixture for predictable runs. I would need a representative corpus, independent evaluation, monitoring, access controls and operational targets before proposing production use.

Interviewer: What did your audit actually change?

You: It added model-aware readiness and request limits, made Kafka wait for confirmed output before acknowledging input, repaired Python 3.12 client compatibility, and added training-fold-only evaluation. Automated checks exercise failures as well as normal predictions. The verification record separates local Python results, real infrastructure runs and mocked Bedrock checks.

2. Explain the machine learning without hiding behind terms

Interviewer: How does TF-IDF help?

You: It turns words into numeric features. A word gets more weight when it appears in this message but is uncommon across the training messages. The vectorizer learns its vocabulary and frequency statistics during fitting. At prediction time it applies that stored mapping, rather than learning from the new message.

> Analogy: if every shop receipt says "item," that word tells me little. A rarer product name helps distinguish one receipt from the rest. TF-IDF gives more weight to those distinguishing terms.

Interviewer: Why logistic regression?

You: It is a fast linear baseline for sparse text features. It learns a weight for each feature and converts the weighted score into class probabilities. That is sufficient for demonstrating serving and streaming. A more complex classifier would not repair missing acknowledgements or false readiness.

Interviewer: Why save a Pipeline?

You: The classifier expects exactly the feature mapping used in training. The Pipeline keeps TF-IDF and logistic regression together, so inference uses the fitted vocabulary and weights. Saving only the final estimator would make it easy to supply incompatible vectors.

Interviewer: You achieved 100% accuracy?

You: That was training accuracy on all 40 fixture records. Five-fold evaluation averaged 77.5% accuracy, with 12.25 percentage points of fold dispersion. Mean spam recall was 65%. I report both so that memorising this tiny training set is not mistaken for generalisation.

Interviewer: Is 77.5% a deployable result?

You: It is a diagnostic on a small curated fixture. Each fold tests only eight messages. Its standard deviation is not a confidence interval, and none of these figures estimates inbox performance reliably. One separate eight-message audit split scored 100%, which illustrates why I would not choose a flattering split as the headline.

Interviewer: Where would leakage occur?

You: If I fitted the vectorizer on the full corpus before cross-validation, held-out messages would influence vocabulary and IDF. Here, cross-validation fits the entire Pipeline separately on each training fold. After evaluation, a fresh Pipeline fits all 40 records to create the serving artifact.

Interviewer: What does the confidence field mean?

You: It is the maximum probability returned by logistic regression. The project has no calibration evaluation. A score of 0.66 is not a validated 66% correctness rate on future messages. A message with no known tokens receives a warning because its prediction is driven by the model's intercept rather than recognised word evidence.

3. Explain the API and the repaired readiness behavior

Interviewer: Why are health and readiness separate?

You: `/health` says the process can answer. `/ready` says it has a usable model. The audit removed the assumption that a green process heartbeat proves inference is available. Missing or corrupt artifacts now make readiness and prediction return 503, while liveness can remain 200.

> Analogy: a restaurant can answer its phone while its oven is broken. Liveness checks the phone; readiness checks whether the kitchen can take orders.

Interviewer: Why change lazy loading?

You: Loading during the first request created a gap between healthy startup and working inference. The lifespan handler now loads and verifies the artifact once before normal requests. Each process still has its own model; this is not shared memory across Kubernetes replicas.

Interviewer: What does validation reject?

You: The API requires one strict string named `text`, with one to 10,000 characters and at least one non-whitespace character. Missing values, nulls, numbers, blank text, excess length and extra fields return 422. It preserves the original message. Unknown-vocabulary text is allowed but clearly marked with `no_known_tokens`.

Interviewer: Are you protected against every harmful request?

You: These checks improve the input contract. They do not supply authentication, rate limits, a global HTTP body-size policy or a production security review. The project does not claim those features.

Interviewer: Can you safely load any uploaded pickle?

You: No. Joblib uses pickle-based serialization, which can execute code. The service loads the trusted artifact produced by its build. A production model supply chain would need provenance, controlled publication and compatible runtime versions.

4. Trace the streaming guarantee

Interviewer: What happens to one message?

You: The producer writes to `messages`. The consumer calls `/predict`, publishes the result to `predictions`, waits for its send future to confirm delivery, then commits that input record's next offset. It disables automatic commits and supplies an explicit partition/offset map.

Interviewer: Why commit offset 18 after processing offset 17?

You: A committed offset names the next record to consume. For partition 2 at offset 17, the consumer confirms the output and commits partition 2 at 18. It does not accidentally acknowledge a later fetched record or another partition.

Interviewer: What was wrong with flush alone?

You: The old flow did not inspect the individual send future, while input offsets were committed automatically. In fault injection, a failed publication was not surfaced by that control flow. Waiting on `send(...).get(...)` observes broker failure, and the commit happens only after success.

Interviewer: Is it exactly once now?

You: No. It is at least once. A crash after output acknowledgement but before input commit can produce the same output again on restart. `messages:2:17` remains the stable event ID on replay. Downstream deduplication could use it, but that deduplicator is not implemented.

Interviewer: What if a message is malformed?

You: The consumer fails without committing it. That makes the failure visible and preserves the record for retry, but can block the partition until an operator intervenes. A dead-letter topic with a documented handling policy would be a future improvement. Silently skipping the message would hide data loss.

Interviewer: What about retries?

You: Connection failures, timeouts, 429 and server errors have bounded retries. A permanent 400 or 422 is not retried as if waiting longer could fix the payload. Persistent failure leaves the input unacknowledged. Retry safety and duplicate handling are separate concerns.

5. Infrastructure questions

Interviewer: What does Docker make reproducible?

You: It packages the application, supported dependencies and trained artifact. The same serving image can run behind Kafka or on Kubernetes. I do not claim eternal binary reproducibility: the base tag and some transitive dependencies can still change. Reproducible behavior needs more than a Dockerfile's presence.

Interviewer: Why does Kafka advertise kafka:9092?

You: That is its name inside the Compose network. Clients learn advertised listener addresses from the broker. Returning localhost to another container would send that client back to itself. A separate host listener would be needed for general host-client use; publishing port 9092 alone does not fix advertised metadata.

Interviewer: What do the Kubernetes probes do?

You: Startup allows time for model loading. Readiness controls whether the pod should receive traffic and checks `/ready`. Liveness checks `/health` to decide whether the process needs restarting. The Deployment requests two replicas; the integration check verifies a deleted pod is replaced.

Interviewer: What is KRaft?

You: Kafka's controllers manage metadata using Raft consensus, replacing the separate ZooKeeper dependency. Our single node runs both broker and controller roles. That simplifies a local demonstration but does not prove controller-quorum or multi-broker fault tolerance.

Interviewer: What is the limit of the Kubernetes test?

You: It verifies local rollout, real HTTP predictions and replacement of a deleted pod. It does not measure cross-node failover, production throughput or a latency service-level objective. Two replicas on one local cluster are not a disaster-recovery result.

Interviewer: Is Terraform provisioning the Kubernetes cluster?

You: No. The supplied Terraform creates only an S3 bucket and DynamoDB table in LocalStack. The local Kubernetes cluster is created separately for the deployment test. The API does not write artifacts or predictions to those cloud resources.

Interviewer: What did your Bedrock test prove?

You: It proved request construction, response parsing and invalid-input rejection with a mocked SDK response. The CLI obtains a real classifier result before making its optional LLM call. There is no live Bedrock result to claim. Even a successful call would supply an interpretation, not measured feature attribution from logistic-regression coefficients.

6. Answer the advanced follow-ups honestly

Interviewer: How would you harden this next?

You: I would first choose representative data and an independent evaluation protocol. For operations, I would define throughput and latency targets, measure consumer lag, add a poison-record policy, design deduplication, authenticate requests and control artifact publication. Each change should have an acceptance check. They are plans, not completed features.

Interviewer: How does this compare with a full MLOps platform?

You: It demonstrates a tested subset: packaging, serving, stream integration, local orchestration and emulated resource provisioning. It has no production monitoring, drift detector, model registry, retraining scheduler, canary rollout or rollback controller. I would present its scope clearly and show the passing execution evidence.

Practice

Explain one Kafka record from offset 17 through output acknowledgement and offset 18 commit. Sketch which modules are connected and which are separate. Explain why 100% training accuracy and green liveness are insufficient. Then reproduce the CLI and the failure tests without memorising the wording above.

Sources: Implementation | Execution evidence | Maintained document index